SQL SUBQUERY CASE STUDY

Car Sales Orders Database

Assignment Submission

Database: orders | Table: Orders | Records: 50 | Questions: 14

1. Introduction

This assignment uses a car-sales Orders database to practise SQL subqueries. The dataset contains 50 purchase records across different customers, products, car categories and cities. The questions represent business requests such as finding above-average purchases, identifying maximum and minimum orders, and comparing category or city performance.

2. Objectives

- Create a relational table with an appropriate primary key and data types.
- Insert and retrieve the 50 supplied car-sales records.
- Use scalar, aggregate and correlated subqueries.
- Apply GROUP BY, HAVING, DISTINCT, MAX, MIN and AVG with subqueries.
- Translate business questions into SQL queries and interpret the results.

3. Database Schema

Database name: orders

Column	Data type	Description
Order_ID	INT	Primary key; unique order identifier
Customer_Name	VARCHAR(100)	Customer's name
Product_Name	VARCHAR(100)	Car model purchased
Car_Category	VARCHAR(50)	SUV, Sedan, Hatchback or MPV
City	VARCHAR(50)	City of the order
Price	DECIMAL(10,2)	Order price in Indian rupees

Total records supplied: 50. Order IDs range from 1001 to 1050.

4. Complete Database Creation and Data Insertion Code

The following is a single, clean, runnable version of the original setup. It preserves the supplied table structure and all 50 supplied records. The uploaded file repeated the CREATE TABLE and INSERT batch; the duplicate copy is intentionally not repeated here.

```
CREATE DATABASE orders;
USE orders;

CREATE TABLE Orders (
    Order_ID INT PRIMARY KEY,
    Customer_Name VARCHAR(100),
    Product_Name VARCHAR(100),
    Car_Category VARCHAR(50),
    City VARCHAR(50),
    Price DECIMAL(10,2)
);

INSERT INTO Orders
    (Order_ID, Customer_Name, Product_Name, Car_Category, City, Price)
VALUES
(1001, 'Aarav Patel', 'Hyundai Creta', 'SUV', 'Ahmedabad', 1450000),
(1002, 'Priya Shah', 'Maruti Swift', 'Hatchback', 'Vadodara', 750000),
(1003, 'Rahul Mehta', 'Tata Nexon', 'SUV', 'Surat', 1200000),
(1004, 'Neha Joshi', 'Honda City', 'Sedan', 'Rajkot', 1350000),
(1005, 'Karan Desai', 'Maruti Baleno', 'Hatchback', 'Ahmedabad', 850000),
(1006, 'Riya Trivedi', 'Mahindra XUV700', 'SUV', 'Vadodara', 2200000),
(1007, 'Arjun Patel', 'Toyota Innova Crysta', 'MPV', 'Surat', 2500000),
(1008, 'Sneha Shah', 'Tata Tiago', 'Hatchback', 'Anand', 600000),
(1009, 'Vivek Parmar', 'Hyundai Verna', 'Sedan', 'Bhavnagar', 1250000),
```

```
(1010, 'Pooja Bhatt', 'Kia Seltos', 'SUV', 'Ahmedabad', 1600000),
(1011, 'Devang Modi', 'Maruti Dzire', 'Sedan', 'Vadodara', 800000),
(1012, 'Anjali Pandya', 'Tata Punch', 'SUV', 'Surat', 750000),
(1013, 'Nirav Shah', 'Hyundai i20', 'Hatchback', 'Rajkot', 900000),
(1014, 'Mihir Patel', 'Toyota Fortuner', 'SUV', 'Ahmedabad', 4200000),
(1015, 'Krisha Mehta', 'Honda Amaze', 'Sedan', 'Anand', 850000),
(1016, 'Yash Thakkar', 'Kia Sonet', 'SUV', 'Vadodara', 1100000),
(1017, 'Dhruv Joshi', 'Maruti WagonR', 'Hatchback', 'Surat', 650000),
(1018, 'Isha Desai', 'Skoda Slavia', 'Sedan', 'Ahmedabad', 1500000),
(1019, 'Manav Shah', 'Mahindra Thar', 'SUV', 'Rajkot', 1800000),
(1020, 'Tanya Patel', 'Toyota Glanza', 'Hatchback', 'Vadodara', 800000),
(1021, 'Parth Mehta', 'MG Hector', 'SUV', 'Surat', 1900000),
(1022, 'Jinal Shah', 'Hyundai Aura', 'Sedan', 'Anand', 750000),
(1023, 'Meet Desai', 'Tata Altroz', 'Hatchback', 'Ahmedabad', 800000),
(1024, 'Aditi Joshi', 'Jeep Compass', 'SUV', 'Vadodara', 2500000),
(1025, 'Rohan Trivedi', 'Maruti Ertiga', 'MPV', 'Surat', 1200000),
(1026, 'Disha Patel', 'Volkswagen Virtus', 'Sedan', 'Rajkot', 1600000),
(1027, 'Sahil Shah', 'Renault Kwid', 'Hatchback', 'Ahmedabad', 500000),
(1028, 'Mansi Mehta', 'Toyota Urban Cruiser', 'SUV', 'Vadodara', 1300000),
(1029, 'Hardik Patel', 'Hyundai Alcazar', 'SUV', 'Surat', 2100000),
(1030, 'Nidhi Desai', 'Honda Elevate', 'SUV', 'Anand', 1500000),
(1031, 'Jay Parmar', 'Maruti Celerio', 'Hatchback', 'Bhavnagar', 600000),
(1032, 'Simran Shah', 'Tata Harrier', 'SUV', 'Ahmedabad', 2200000),
(1033, 'Om Patel', 'Hyundai Grand i10', 'Hatchback', 'Vadodara', 650000),
(1034, 'Khushi Mehta', 'Skoda Kushaq', 'SUV', 'Surat', 1400000),
(1035, 'Rakesh Joshi', 'Maruti Ciaz', 'Sedan', 'Rajkot', 1000000),
(1036, 'Diya Trivedi', 'Kia Carens', 'MPV', 'Ahmedabad', 1400000),
(1037, 'Akash Shah', 'Tata Safari', 'SUV', 'Vadodara', 2500000),
(1038, 'Palak Desai', 'Maruti Fronx', 'SUV', 'Surat', 950000),
(1039, 'Naitik Patel', 'Hyundai Exter', 'SUV', 'Anand', 800000),
(1040, 'Sonal Mehta', 'Honda City', 'Sedan', 'Ahmedabad', 1400000),
(1041, 'Krunal Shah', 'Mahindra Scorpio N', 'SUV', 'Vadodara', 2000000),
(1042, 'Hetal Joshi', 'Maruti Swift', 'Hatchback', 'Surat', 780000),
(1043, 'Amit Desai', 'Toyota Innova Hycross', 'MPV', 'Ahmedabad', 2800000),
(1044, 'Riddhi Patel', 'Tata Nexon', 'SUV', 'Rajkot', 1250000),
(1045, 'Suresh Mehta', 'Hyundai Verna', 'Sedan', 'Vadodara', 1300000),
(1046, 'Bhavya Shah', 'Kia Seltos', 'SUV', 'Anand', 1650000),
(1047, 'Chirag Parmar', 'Maruti Baleno', 'Hatchback', 'Surat', 870000),
(1048, 'Esha Trivedi', 'Mahindra XUV700', 'SUV', 'Ahmedabad', 2250000),
(1049, 'Tirth Patel', 'Toyota Fortuner', 'SUV', 'Vadodara', 4300000),
(1050, 'Nisha Desai', 'Maruti Dzire', 'Sedan', 'Bhavnagar', 820000);

SELECT * FROM Orders;
```

5. SQL Subquery Questions and Solutions

The uploaded file contains 14 distinct question prompts. The solutions below follow the written business requirement. Where an attempted query in the source did not match its prompt, the solution is corrected and the difference is noted.

1. Orders Above the Average for Their Category

Business question: The business wants to identify orders priced above the average price of the same car category.

SQL query:

```
SELECT o.Order_ID, o.Customer_Name, o.Product_Name, o.Car_Category, o.City, o.Price
FROM Orders AS o
WHERE o.Price > (
    SELECT AVG(i.Price)
    FROM Orders AS i
    WHERE i.Car_Category = o.Car_Category
```

```
)
ORDER BY o.Car_Category, o.Price DESC;
```

Explanation: The inner query calculates the average price for the outer row's category. The outer query returns only orders whose price is greater than that category average.

Expected result: The result contains each order whose price exceeds the average for its own category.

Result preview (25 row(s)):

Order_ID	Customer_Name	Product_Name	Car_Category	City	Price
1013	Nirav Shah	Hyundai i20	Hatchback	Rajkot	₹900,000.00
1047	Chirag Parmar	Maruti Baleno	Hatchback	Surat	₹870,000.00
1005	Karan Desai	Maruti Baleno	Hatchback	Ahmedabad	₹850,000.00
1020	Tanya Patel	Toyota Glanza	Hatchback	Vadodara	₹800,000.00
1023	Meet Desai	Tata Altroz	Hatchback	Ahmedabad	₹800,000.00
1042	Hetal Joshi	Maruti Swift	Hatchback	Surat	₹780,000.00
1002	Priya Shah	Maruti Swift	Hatchback	Vadodara	₹750,000.00
1043	Amit Desai	Toyota Innova Hycross	MPV	Ahmedabad	₹2,800,000.00
1007	Arjun Patel	Toyota Innova Crysta	MPV	Surat	₹2,500,000.00
1049	Tirth Patel	Toyota Fortuner	SUV	Vadodara	₹4,300,000.00

Showing the first 10 of 25 result rows. Run the query in MySQL to view the complete result.

2. Highest-Priced SUV in Each City

Business question: Management wants the most expensive SUV purchased in each city, with customer, city, product and price.

SQL query:

```
SELECT o.Customer_Name, o.City, o.Product_Name, o.Price
FROM Orders AS o
WHERE o.Car_Category = 'SUV'
  AND o.Price = (
      SELECT MAX(i.Price)
      FROM Orders AS i
      WHERE i.City = o.City
        AND i.Car_Category = 'SUV'
  )
ORDER BY o.City;
```

Explanation: The correlated subquery finds the maximum SUV price in the current row's city. Restricting both the outer and inner queries to SUV ensures non-SUV orders do not affect the result.

Expected result: One highest-priced SUV order is returned for every city that has an SUV order. Ties are retained.

Result preview (5 row(s)):

Customer_Name	City	Product_Name	Price
Mihir Patel	Ahmedabad	Toyota Fortuner	₹4,200,000.00
Bhavya Shah	Anand	Kia Seltos	₹1,650,000.00
Manav Shah	Rajkot	Mahindra Thar	₹1,800,000.00
Hardik Patel	Surat	Hyundai Alcazar	₹2,100,000.00
Tirth Patel	Vadodara	Toyota Fortuner	₹4,300,000.00

3. Customers With an Above-Average Car for Their City and Category

Business question: Find customers whose car price is above the average for the same city and car-category combination. Return customer, city, category, product and price.

SQL query:

```
SELECT o.Customer_Name, o.City, o.Car_Category, o.Product_Name, o.Price
FROM Orders AS o
WHERE o.Price > (
    SELECT AVG(i.Price)
    FROM Orders AS i
    WHERE i.City = o.City
      AND i.Car_Category = o.Car_Category
)
ORDER BY o.City, o.Car_Category, o.Price DESC;
```

Explanation: The subquery is correlated on both City and Car_Category. This compares each order with the average for its own city-category group, matching the written business requirement.

Expected result: The output contains orders above their city-and-category average. This corrects the supplied attempt, which compared only the city average.

Result preview (24 row(s)):

Customer_Name	City	Car_Category	Product_Name	Price
Karan Desai	Ahmedabad	Hatchback	Maruti Baleno	₹850,000.00
Meet Desai	Ahmedabad	Hatchback	Tata Altroz	₹800,000.00
Amit Desai	Ahmedabad	MPV	Toyota Innova Hycross	₹2,800,000.00
Mihir Patel	Ahmedabad	SUV	Toyota Fortuner	₹4,200,000.00
Isha Desai	Ahmedabad	Sedan	Skoda Slavia	₹1,500,000.00
Bhavya Shah	Anand	SUV	Kia Seltos	₹1,650,000.00
Nidhi Desai	Anand	SUV	Honda Elevate	₹1,500,000.00
Krisha Mehta	Anand	Sedan	Honda Amaze	₹850,000.00
Vivek Parmar	Bhavnagar	Sedan	Hyundai Verna	₹1,250,000.00
Manav Shah	Rajkot	SUV	Mahindra Thar	₹1,800,000.00

Showing the first 10 of 24 result rows. Run the query in MySQL to view the complete result.

4. Highest-Priced Order for Each City and Car Category

Business question: Find the highest-priced order within each City + Car Category combination. Return city, category, customer, product and price.

SQL query:

```
SELECT o.City, o.Car_Category, o.Customer_Name, o.Product_Name, o.Price
FROM Orders AS o
WHERE o.Price = (
    SELECT MAX(i.Price)
    FROM Orders AS i
    WHERE i.City = o.City
      AND i.Car_Category = o.Car_Category
)
ORDER BY o.City, o.Car_Category;
```

Explanation: The inner query calculates the maximum price for the current row's city and category. The outer query returns the row or rows matching that maximum.

Expected result: The output contains the top-priced order for every city-category group, including ties.

Result preview (18 row(s)):

City	Car_Category	Customer_Name	Product_Name	Price
Ahmedabad	Hatchback	Karan Desai	Maruti Baleno	₹850,000.00
Ahmedabad	MPV	Amit Desai	Toyota Innova Hycross	₹2,800,000.00
Ahmedabad	SUV	Mihir Patel	Toyota Fortuner	₹4,200,000.00
Ahmedabad	Sedan	Isha Desai	Skoda Slavia	₹1,500,000.00
Anand	Hatchback	Sneha Shah	Tata Tiago	₹600,000.00
Anand	SUV	Bhavya Shah	Kia Seltos	₹1,650,000.00
Anand	Sedan	Krisha Mehta	Honda Amaze	₹850,000.00
Bhavnagar	Hatchback	Jay Parmar	Maruti Celerio	₹600,000.00
Bhavnagar	Sedan	Vivek Parmar	Hyundai Verna	₹1,250,000.00
Rajkot	Hatchback	Nirav Shah	Hyundai i20	₹900,000.00

Showing the first 10 of 18 result rows. Run the query in MySQL to view the complete result.

5. Customers Whose Purchase Is the Highest in Their City

Business question: Show customers whose order represents the maximum sales value in their city. Return customer, city, product, category and price.

SQL query:

```
SELECT o.Customer_Name, o.City, o.Product_Name, o.Car_Category, o.Price
FROM Orders AS o
WHERE o.Price = (
    SELECT MAX(i.Price)
    FROM Orders AS i
    WHERE i.City = o.City
)
ORDER BY o.City;
```

Explanation: For each order, the correlated subquery finds the maximum order price in that order's city. Only matching maximum-price orders are displayed.

Expected result: The output identifies the highest-value order or tied orders in each city.

Result preview (6 row(s)):

Customer_Name	City	Product_Name	Car_Category	Price
Mihir Patel	Ahmedabad	Toyota Fortuner	SUV	₹4,200,000.00
Bhavya Shah	Anand	Kia Seltos	SUV	₹1,650,000.00
Vivek Parmar	Bhavnagar	Hyundai Verna	Sedan	₹1,250,000.00
Manav Shah	Rajkot	Mahindra Thar	SUV	₹1,800,000.00
Arjun Patel	Surat	Toyota Innova Crysta	MPV	₹2,500,000.00
Tirth Patel	Vadodara	Toyota Fortuner	SUV	₹4,300,000.00

6. Identify the Highest-Value Order

Business question: Identify the order with the greatest price in the entire database.

SQL query:

```
SELECT Order_ID, Customer_Name, Product_Name, City, Price
FROM Orders
WHERE Price = (SELECT MAX(Price) FROM Orders);
```

Explanation: The subquery returns the single maximum price in the full Orders table. The outer query retrieves the order details with that price.

Expected result: The output shows the most expensive order in the dataset.

Result preview (1 row(s)):

Order_ID	Customer_Name	Product_Name	City	Price
1049	Tirth Patel	Toyota Fortuner	Vadodara	₹4,300,000.00

7. Orders Above the Overall Average Price

Business question: Return customer name, product name, city and price for orders priced above the average of all orders.

SQL query:

```
SELECT Customer_Name, Product_Name, City, Price
FROM Orders
WHERE Price > (SELECT AVG(Price) FROM Orders)
ORDER BY Price DESC;
```

Explanation: The scalar subquery calculates one overall average. Every order is compared with that value.

Expected result: The output lists all orders whose price is greater than the overall dataset average.

Result preview (19 row(s)):

Customer_Name	Product_Name	City	Price
Tirth Patel	Toyota Fortuner	Vadodara	₹4,300,000.00
Mihir Patel	Toyota Fortuner	Ahmedabad	₹4,200,000.00
Amit Desai	Toyota Innova Hycross	Ahmedabad	₹2,800,000.00
Arjun Patel	Toyota Innova Crysta	Surat	₹2,500,000.00
Aditi Joshi	Jeep Compass	Vadodara	₹2,500,000.00
Akash Shah	Tata Safari	Vadodara	₹2,500,000.00
Esha Trivedi	Mahindra XUV700	Ahmedabad	₹2,250,000.00
Riya Trivedi	Mahindra XUV700	Vadodara	₹2,200,000.00
Simran Shah	Tata Harrier	Ahmedabad	₹2,200,000.00
Hardik Patel	Hyundai Alcazar	Surat	₹2,100,000.00

Showing the first 10 of 19 result rows. Run the query in MySQL to view the complete result.

8. Orders Above Ahmedabad's Average

Business question: Find orders whose price is higher than the average order price in Ahmedabad. Return customer, city, product and price.

SQL query:

```
SELECT Customer_Name, City, Product_Name, Price
FROM Orders
WHERE Price > (
    SELECT AVG(Price)
    FROM Orders
    WHERE City = 'Ahmedabad'
)
ORDER BY Price DESC;
```

Explanation: The subquery calculates the average using only Ahmedabad orders. The outer query compares every order in the table against that Ahmedabad benchmark.

Expected result: The result can include orders from any city, provided their price is above Ahmedabad's average.

Result preview (13 row(s)):

Customer_Name	City	Product_Name	Price
Tirth Patel	Vadodara	Toyota Fortuner	₹4,300,000.00
Mihir Patel	Ahmedabad	Toyota Fortuner	₹4,200,000.00
Amit Desai	Ahmedabad	Toyota Innova Hycross	₹2,800,000.00
Arjun Patel	Surat	Toyota Innova Crysta	₹2,500,000.00
Aditi Joshi	Vadodara	Jeep Compass	₹2,500,000.00
Akash Shah	Vadodara	Tata Safari	₹2,500,000.00
Esha Trivedi	Ahmedabad	Mahindra XUV700	₹2,250,000.00
Riya Trivedi	Vadodara	Mahindra XUV700	₹2,200,000.00
Simran Shah	Ahmedabad	Tata Harrier	₹2,200,000.00
Hardik Patel	Surat	Hyundai Alcazar	₹2,100,000.00

Showing the first 10 of 13 result rows. Run the query in MySQL to view the complete result.

9. Find the Cheapest SUV

Business question: Identify the SUV with the lowest purchase price. Return customer, product, city and price.

SQL query:

```
SELECT Customer_Name, Product_Name, City, Price
FROM Orders
WHERE Car_Category = 'SUV'
  AND Price = (
      SELECT MIN(Price)
      FROM Orders
      WHERE Car_Category = 'SUV'
  );
```

Explanation: The subquery finds the minimum price among SUVs only. The outer query returns the SUV order or orders at that minimum price.

Expected result: The output identifies the least expensive SUV purchase in the dataset.

Result preview (1 row(s)):

Customer_Name	Product_Name	City	Price
Anjali Pandya	Tata Punch	Surat	₹750,000.00

10. Cities With High-Value Sales

Business question: A high-value sale is an order priced above the average price of all orders. Find cities with at least one such sale.

SQL query:

```
SELECT DISTINCT City
FROM Orders
WHERE Price > (SELECT AVG(Price) FROM Orders)
ORDER BY City;
```

Explanation: The subquery calculates the overall average. The outer query filters high-value orders and DISTINCT ensures each city appears only once.

Expected result: The output is a distinct list of cities with one or more above-average orders.

Result preview (5 row(s)):

City
Ahmedabad

Anand
Rajkot
Surat
Vadodara

11. Orders More Expensive Than the Average Sedan

Business question: Return customer name, product name, category and price for all orders priced above the average Sedan price.

SQL query:

```
SELECT Customer_Name, Product_Name, Car_Category, Price
FROM Orders
WHERE Price > (
    SELECT AVG(Price)
    FROM Orders
    WHERE Car_Category = 'Sedan'
)
ORDER BY Price DESC;
```

Explanation: The subquery calculates the average price of Sedan orders. The outer query compares all orders—not only Sedans—with that benchmark.

Expected result: The output contains every order more expensive than the average Sedan.

Result preview (29 row(s)):

Customer_Name	Product_Name	Car_Category	Price
Tirth Patel	Toyota Fortuner	SUV	₹4,300,000.00
Mihir Patel	Toyota Fortuner	SUV	₹4,200,000.00
Amit Desai	Toyota Innova Hycross	MPV	₹2,800,000.00
Arjun Patel	Toyota Innova Crysta	MPV	₹2,500,000.00
Aditi Joshi	Jeep Compass	SUV	₹2,500,000.00
Akash Shah	Tata Safari	SUV	₹2,500,000.00
Esha Trivedi	Mahindra XUV700	SUV	₹2,250,000.00
Riya Trivedi	Mahindra XUV700	SUV	₹2,200,000.00
Simran Shah	Tata Harrier	SUV	₹2,200,000.00
Hardik Patel	Hyundai Alcazar	SUV	₹2,100,000.00

Showing the first 10 of 29 result rows. Run the query in MySQL to view the complete result.

12. Orders Above the Average Price of Cars Costing More Than ₹20 Lakhs

Business question: Find customers whose order price is greater than the average price of orders above ₹20,00,000. Return customer, product and price.

SQL query:

```
SELECT Customer_Name, Product_Name, Price
FROM Orders
WHERE Price > (
    SELECT AVG(Price)
    FROM Orders
    WHERE Price > 2000000
)
ORDER BY Price DESC;
```

Explanation: The inner query first selects orders above ₹20 lakh and calculates their average. The outer query returns orders whose price is greater than that calculated average.

Expected result: The result is based on the average of the over-₹20-lakh subset, not simply every order above ₹20 lakh.

Result preview (3 row(s)):

Customer_Name	Product_Name	Price
Tirth Patel	Toyota Fortuner	₹4,300,000.00
Mihir Patel	Toyota Fortuner	₹4,200,000.00
Amit Desai	Toyota Innova Hycross	₹2,800,000.00

13. Find the City With the Highest-Priced Order

Business question: Identify the city, customer, product and price for the most expensive order in the dataset.

SQL query:

```
SELECT City, Customer_Name, Product_Name, Price
FROM Orders
WHERE Price = (SELECT MAX(Price) FROM Orders);
```

Explanation: The scalar subquery finds the maximum order price in the whole table. The outer query returns the city and details of the matching order.

Expected result: If multiple orders share the maximum price, all tied rows are returned.

Result preview (1 row(s)):

City	Customer_Name	Product_Name	Price
Vadodara	Tirth Patel	Toyota Fortuner	₹4,300,000.00

14. Categories With Above-Average Orders

Business question: Return the category and its average price for categories whose average is higher than the overall average order price.

SQL query:

```
SELECT Car_Category, AVG(Price) AS Average_Category_Price
FROM Orders
GROUP BY Car_Category
HAVING AVG(Price) > (SELECT AVG(Price) FROM Orders)
ORDER BY Average_Category_Price DESC;
```

Explanation: GROUP BY calculates one average per category. HAVING filters those grouped averages against the overall average returned by the subquery.

Expected result: The output lists only categories whose average price exceeds the average price across all orders.

Result preview (2 row(s)):

Car_Category	Average_Category_Price
MPV	₹1,975,000.00
SUV	₹1,865,217.39

6. SQL Concepts Used

Concept	Meaning in this assignment
Subquery	A query nested inside another SQL statement.
Scalar subquery	A subquery that returns one value, such as AVG(Price) or MAX(Price).

Correlated subquery	A subquery that refers to a column from the outer query and is evaluated for the outer row/group.
Aggregate functions	AVG(), MAX() and MIN() calculate summary values.
GROUP BY	Groups records so an aggregate can be calculated for each category.
HAVING	Filters grouped results after aggregation.
DISTINCT	Removes duplicate values, such as repeated city names.

7. Key Observations

The overall average order price in the supplied dataset is ₹1,443,400.00.

The highest-priced order is Toyota Fortuner purchased by Tirth Patel in Vadodara at ₹4,300,000.00.

The lowest-priced SUV order is Tata Punch purchased by Anjali Pandya in Surat at ₹750,000.00.

Average price by category:

Category	Average price
MPV	₹1,975,000.00
SUV	₹1,865,217.39
Sedan	₹1,147,272.73
Hatchback	₹729,166.67

8. Conclusion

This case study demonstrates how SQL subqueries can answer practical business questions using a car-sales database. Scalar subqueries provide overall benchmark values, while correlated subqueries compare each order with its own city, category or city-category group. Aggregate functions and grouping make it possible to identify high-value orders and compare average prices. The assignment solutions are based on the 50 records and the 14 distinct prompts present in the uploaded SQL file.

9. Source File Note

Source used: SQL SUBQUERY.sql supplied for this assignment. The file includes 50 data records and 14 distinct business-question prompts. It does not contain 50 separate question prompts. Duplicate setup statements were consolidated, and queries were aligned with the written prompts where the source attempts contained syntax or logic errors.